NON-STATIONARITY AS LEVELS: UNSUPERVISED ENVIRONMENT DESIGN FOR ROBUST REINFORCEMENT LEARNING

Jeremias Ferrao & Matthijs Van Der Lende
University of Groningen
Groningen, The Netherlands

ABSTRACT

Reinforcement learning agents are usually trained and evaluated under stationary dynamics, yet deployed systems face drift: parameters shift mid-episode and the agent must cope with non-stationarity it never perceived during training. We approach the NS-Gym competition setting by casting each non-stationary configuration as a level and applying Unsupervised Environment Design on top of PPO. Prioritized Level Replay, together with ACCEL-style mutations of high-scoring configurations, continually targets the levels the agent handles worst. A single model-free policy is trained per task and paired with an inference path optimized for low-latency CPU execution. On the held-out competition benchmark our submission ranks first on all four evaluation axes, performance, adaptability, resilience, and efficiency, simultaneously, making it the only non-dominated entry, while competing teams trade these objectives against one another and specialize a separate method for each environment. We make our code available at https://github.com/Jazhyc/nsgym-solution.

1 INTRODUCTION

Reinforcement learning (RL) agents are typically trained and evaluated under the assumption that the environment dynamics are stationary. In practice, agents are deployed in systems whose dynamics drift: actuators wear and weaken, payloads change a robot's mass, joints accumulate friction, and surfaces grow slick with moisture or use. A policy that was optimal at deployment slowly ceases to be. The NS-Gym competition (Keplinger et al., 2025) is a challenge for RL agents under non-stationarity: agents must learn CartPole, FrozenLake, and Ant while physics parameters change mid-episode.

The core difficulty is an instance of the plasticity-stability dilemma studied in continual learning (CL; McCloskey & Cohen, 1989): a policy must remain flexible enough to learn new information (plasticity) while retaining what it already knows (stability). Standard CL solutions, such as elastic weight consolidation (EWC; Kirkpatrick et al., 2017), protect prior knowledge at task boundaries by regularizing weights using the Fisher information matrix. However, in our setting there are no clear boundaries: dynamics can drift continually within episodes, and a policy must be robust to these changes. Adaptation methods such as change-point detectors (Padakandla et al., 2020) and meta-learners (Rakelly et al., 2019) sidestep this by reacting to each shift, but require either a detectable signal or enough within-episode transitions to re-identify the dynamics. Domain randomization (Tobin et al., 2017) instead achieves plasticity by training over many configurations simultaneously, but samples that distribution uniformly, spending equal time on configurations the policy already handles and on those it still struggles with.

We address the generalization problem through training distribution design. We represent a complete non-stationarity specification (which parameters drift, when they change via the *scheduler*, and how they change via the *update function*) as a *level* in the sense of unsupervised environment design (UED; Dennis et al., 2020). In place of uniform domain randomization, prioritized level replay (PLR; Jiang et al., 2021) concentrates training on the configurations the current policy handles worst, and Adversarially Compounding Complexity by Editing Levels

(ACCEL; Parker-Holder et al., 2022) evolves new levels from the hardest ones. A single PPO (Schulman et al., 2017) policy trained this way is submitted unchanged, regardless of how much information about each change the agent receives.

On the held-out competition benchmark, this single recipe is the only submission to rank first on every scored axis at once (Section 4.1). Our contributions are:

- A level representation for non-stationarity, pairing six schedulers with up to eleven update functions per parameter to cover gradual drift, stochastic diffusion, oscillation, and transition-distribution shifts (Section 3.3, Appendix C).
- A PLR+ACCEL curriculum over this level space with per-worker independent buffers requiring no inter-process synchronization, reaching approximately 8,000 environment steps per second on Ant (Section 3.4).
- An inference pipeline that reduces per-step policy latency 23–48$\times$ over a standard TorchRL actor by eliminating Python dispatch overhead, making the environment step the bottleneck on every task (Section 3.5).
- A single notification-agnostic policy per task that is competitive across all three notification levels without per-level retraining (Section 3.1).

2 Related Work

Continual learning. Continual learning (McCloskey & Cohen, 1989; van de Ven et al., 2025) studies learning incrementally from a non-stationary data stream. In continual RL (Khetarpal et al., 2022; Abel et al., 2023) the tasks are e.g. shifting environment configurations. Weight-regularization (EWC; Kirkpatrick et al., 2017) and capacity-allocation methods (PackNet; Mallya & Lazebnik, 2018) protect prior knowledge at discrete task boundaries, and so need identifiable boundaries and a phase per task. We share the problem, one policy across changing environment dynamics, but intervene on the training distribution rather than the architecture or objective.

Adaptive approaches. A second family adapts online. Change-point detectors (Padakandla et al., 2020; da Silva et al., 2006) relearn after a detected shift, DaCoRL (Zhang et al., 2023) infers a context and switches network heads, and meta-learners such as PEARL (Rakelly et al., 2019) and VariBAD (Zintgraf et al., 2020) re-identify the task from within-episode experience. All need shifts to be detectable or episodes long enough to supply a signal, neither of which holds when dynamics drift continuously and the agent receives no signal that a change has occurred. We instead require no test-time adaptation.

Robust training. Domain randomization (Tobin et al., 2017) trains over a fixed distribution of configurations; robust MDPs (Nilim & El Ghaoui, 2005) and adversarial training (Pinto et al., 2017) optimize a worst case. We keep the full distribution but reweight it adaptively toward the configurations the current policy handles least well.

Unsupervised environment design. UED treats environment generation as a teacher–student game. PLR (Jiang et al., 2021) curates a buffer of high-scoring levels from random search, with ties to minimax regret (Jiang et al., 2022), and ACCEL (Parker-Holder et al., 2022) mutates high-scoring parents to grow complexity at the frontier. Recent work questions the replay score itself: PLR's value-loss signal is a coarse regret proxy, and both learnability-based selection (Rutherford et al., 2024) and transition-aware regret (Cho et al., 2026) improve on it. We apply PLR and ACCEL to a new design space, non-stationarity *schedules* rather than spatial layouts, and score levels with the standard mean-absolute-advantage signal as a cheap proxy for learning potential. This scoring choice is orthogonal to our core contribution of casting non-stationarity as a level space: the learnability-based and transition-aware regret scores could replace it without further change.

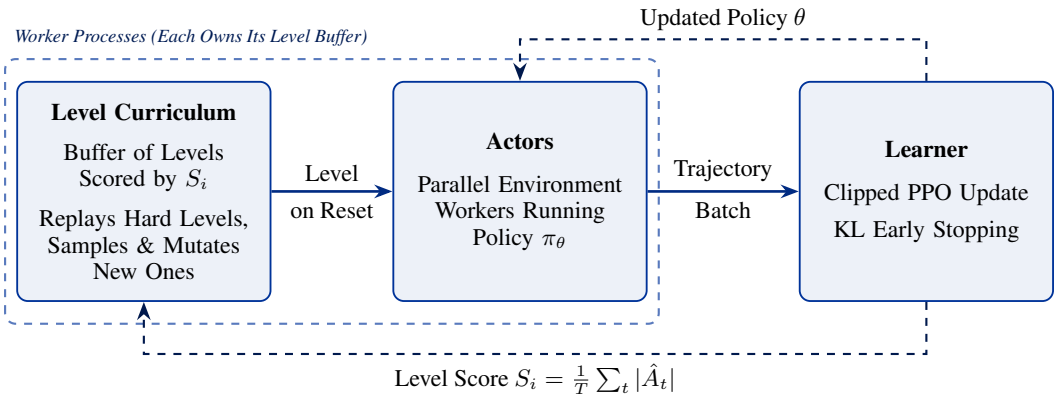


Figure 1: Overview of our unsupervised environment design loop. A *level curriculum* maintains a PLR buffer of scored levels and proposes the next non-stationary configuration, either replaying a level the policy currently handles worst or generating a new one by random sampling or ACCEL mutation. Parallel *actors* collect trajectories under the current policy π_θ. The *learner* computes GAE advantages and a clipped PPO update with KL early stopping, returns the new policy to the actors, and feeds each completed episode's mean absolute advantage back as that level's score (Equation 1). The curriculum is replicated inside every worker, eliminating inter-process communication.

3 Methods

3.1 Competition environments

The competition evaluates agents on three NS-Gym tasks (Keplinger et al., 2025) built on Gymnasium (Towers et al., 2024): CartPole, FrozenLake, and Ant. NS-Gym perturbs environment parameters while an episode unfolds, so the transition dynamics drift over time. CartPole exposes the pole mass, pole length, cart mass, and gravity as tunable parameters; Ant exposes gravity and the torso mass; FrozenLake exposes the transition distribution (p_1, p_2, p_3) over the intended and two lateral movement directions, which starts at the uniform (fully slippery) distribution. FrozenLake's discrete state is one-hot encoded before being passed to the policy.

Agents are evaluated under three notification levels. Under full notification the agent is told when a parameter changes and the magnitude of the change, under change notification it is only told that a change occurred, and under no notification it receives no signal at all. We train a single notification-agnostic policy per task and submit it at all three levels.

3.2 Policy optimization

All agents are trained with PPO (Schulman et al., 2017) and generalized advantage estimation (GAE; Schulman et al., 2016). The actor and critic are separate two-layer multilayer perceptrons, with a categorical policy on the discrete tasks and a squashed Gaussian on Ant. Most hyperparameters follow the defaults of the Stable-Baselines3 PPO implementation (Raffin et al., 2021), with the full per-task settings listed in Appendix A. We describe here only the choices that depart from those defaults to suit the non-stationary setting.

To keep updates stable under a shifting task distribution, the inner optimization epochs stop early once the mean approximate KL divergence between the current and behavior policies exceeds 1.5 times a target threshold. Observation normalization statistics are estimated from random rollouts before training and then frozen, so the input scaling does not itself become a source of non-stationarity.

3.3 Non-stationarity as levels

Our central design decision is to treat a complete non-stationarity specification as a *level* in the sense of UED (Dennis et al., 2020). A level assigns to each tunable parameter of a task a *scheduler*, which determines when the parameter changes, and an *update function*, which determines how it changes,

together with the numeric arguments of both. New levels are drawn by sampling the scheduler and update function uniformly from per-parameter eligible sets and their arguments uniformly from calibrated ranges; some parameters are included only with a fixed probability, so levels also differ in which parameters drift at all. For FrozenLake, the initial transition distribution is itself sampled so that training covers the full range from near-deterministic to fully slippery dynamics. The complete level space is given in Appendix C; its eligible sets and ranges were chosen by intuition about each task's physics and were not tuned empirically.

Casting non-stationarity this way lets us replace uniform domain randomization with an adaptive curriculum based on PLR (Jiang et al., 2021). PLR maintains a buffer of up to N previously played levels, where each level i carries a score S_i estimating its learning potential. After every episode on level i, the score is updated from the GAE advantages $\hat{A}_t$ of that episode,

$$S_i \leftarrow (1-\alpha)\, S_i + \alpha \cdot \frac{1}{T} \sum_{t=1}^{T} \left|\hat{A}_t\right|, \qquad \alpha = \frac{1}{n_i + 1}, \tag{1}$$

where T is the episode length and n_i the number of previous visits, so S_i is the running mean of the per-episode mean absolute advantage. A high score means the critic still misjudges the level and there is room to learn from it.

At each episode boundary, with replay probability p_r a buffered level is replayed, and otherwise a new one is explored; p_r thus sets the balance between revisiting known levels and discovering new ones. A replayed level is drawn from a mixture of two distributions: a scoring distribution P_S that favors high-scoring levels, those the policy can still learn the most from, and a staleness distribution P_C that favors levels not visited for many episodes. The two are combined into the probability $P_{\text{replay}}(i)$ of selecting level i for replay,

$$P_{\text{replay}}(i) = (1-\rho)\, P_S(i) + \rho\, P_C(i), \qquad P_S(i) \propto \left(\frac{1}{r_i}\right)^{1/\beta}, \qquad P_C(i) \propto c_i, \tag{2}$$

where r_i is the rank of S_i among all buffered scores in descending order (so the highest-scoring level has rank 1), β is a temperature controlling how greedily high-scoring levels are favored, and c_i counts the episodes since level i was last played. Each of P_S and P_C is normalized separately over the buffer to sum to one, so their convex combination P_{replay} is itself a valid distribution. The staleness term, weighted by ρ, ensures that no level goes unvisited for so long that its score no longer reflects the current policy. When the buffer is full, newly explored levels evict the lowest-scoring entry.

On top of PLR we add ACCEL-style mutations (Parker-Holder et al., 2022). Random exploration must stumble onto difficult configurations by chance, whereas a level close to a difficult one tends to be difficult as well; mutating high-scoring parents therefore breeds new informative levels and grows difficulty at the frontier of the policy's competence. Concretely, in a fraction p_m of exploration steps, rather than sampling a fresh level, we perturb a parent level drawn from Equation 2. Each numeric argument v of the parent's schedulers and update functions, with valid range $[l, h]$, is perturbed as

$$v' = \text{clip}\left(v + \varepsilon,\, l,\, h\right), \qquad \varepsilon \sim \mathcal{N}\left(0,\, \sigma_m^2 (h-l)^2\right), \tag{3}$$

where σ_m scales the noise relative to each argument's range width; integer arguments are rounded, and the scheduler and update function types themselves are preserved. Unlike the original method we use no explicit regret threshold for accepting mutations: invalid values are prevented by the clipping in Equation 3, and unproductive mutants are simply evicted once their scores fall. Mutation is enabled for all three tasks. All curriculum hyperparameters are listed in Appendix A.

3.4 Training infrastructure

Figure 1 gives an overview of the full training loop. We run training and evaluation entirely on CPU. This is a deliberate choice rather than a competition requirement: the policy is a small multilayer perceptron whose throughput is bounded by the environment step rather than by the network, so a GPU offers little benefit at our batch sizes and adds host-device transfer overhead. Throughput engineering on CPU therefore matters. Training uses TorchRL (Bou et al., 2024) with an asynchronous collector running two groups of twelve parallel environment workers: one group collects while gradients are computed on the other group's batch, bounding policy staleness to a single update. Each

worker process owns an independent PLR buffer, which avoids all inter-process synchronization; since a level is only a small configuration object, holding duplicate buffers adds minimal memory overhead and simplifies the implementation. The learner, the central process that computes GAE and gradients on the collected batches, writes each worker's mean absolute advantage to shared memory; the worker then reads its slot at the next episode boundary to score the level it just finished. This setup reaches roughly 8,000 environment steps per second on Ant on a 12-core desktop CPU; full hardware and software specifications are given in Appendix A.

The stock NS-Gym wrappers also carry per-step overhead that we removed by rewriting them, with the resulting latencies reported in Table 1. The stock classic-control wrapper rebuilds a dictionary observation and its associated change metadata on every step, even when nothing changes; our version applies parameter updates in place on the underlying environment and skips this reconstruction. The stock FrozenLake wrapper is costlier still. Its transition table maps every state-action pair to a distribution over the reachable next states, and when the slipperiness drifts the wrapper regenerates this entire nested structure from scratch, even though only the probabilities change while the set of reachable next states stays fixed. Ours precomputes that fixed structure once at initialization and, on each change, overwrites only the three-element probability vector that actually moves.

	CartPole	FrozenLake
Stock NS-Gym wrapper	11.3	171.8
Rewritten wrapper	8.0	38.2
Plain stationary environment	5.4	4.7
Wrapper speedup	$1.4\times$	$4.5\times$

Table 1: Mean per-step latency in microseconds of the training environments under the stock NS-Gym wrappers and our rewritten ones, with the stationary environment as a lower bound. Measured over 10000 random-action steps with a continuous scheduler, so a parameter update fires every step, the worst case for wrapper overhead. The remaining gap to the plain environments is the cost of the parameter update itself and, for FrozenLake, of sampling transitions from the updated distribution.

Progress is monitored on a held-out set of evaluation levels sampled once from a dedicated seed and kept fixed throughout training. We track both the mean return and the interquartile mean (IQM), which is more robust to outlier levels.

3.5 Inference optimization

The competition evaluator steps a single environment at a time, so policy inference runs at batch size one, where per-step Python dispatch overhead dominates the cost of a small network. We therefore optimized the evaluation path in four successive steps, whose measured latencies are reported in Table 2. First, the TorchRL actor is bypassed and its underlying network is called directly, skipping the dictionary bookkeeping (distribution parameters, log probabilities) that training needs but evaluation does not; actions are sampled with the Gumbel-max trick for the discrete policies and the reparameterized squashed Gaussian for Ant. Second, the network is compiled with the PyTorch JIT, which yields only a marginal gain at batch size one. Third, because the JIT still leaves the PyTorch dispatch in place, we replace the forward pass altogether: the network weights are exported once as contiguous NumPy arrays and the forward runs as a sequence of preallocated matrix multiplications. This NumPy forward supersedes the JIT-compiled one rather than stacking with it, removing the per-layer PyTorch dispatch entirely. Finally, the frozen observation normalization is folded into the first layer's weights and bias, and the raw observation is copied directly into a preallocated single-precision buffer, so a step never touches PyTorch at all. With frozen per-feature mean μ and standard deviation σ, the first layer computes $W_0\big((x-\mu)/\sigma\big) + b_0$ on the normalized input; since this is affine in the raw observation x, it collapses into an equivalent linear layer whose weights W_0/σ absorb the per-feature scaling and whose bias absorbs the mean shift μ, so normalization adds no runtime cost.

Together these optimizations reduce latency by 23X (Ant) to 48X (FrozenLake). The benefit at the system level depends on the cost of the environment itself, which Table 2 also reports: the policy's

share of total step time drops from 64% to 7% on Ant, from 96% to 34% on CartPole, and from 35% to 1% on FrozenLake. After optimization, the environment is the bottleneck on every task.

	Ant	CartPole	FrozenLake
Full TorchRL actor (baseline)	467.5 (64%)	294.7 (96%)	302.5 (35%)
+ direct network call	114.7 (31%)	58.4 (82%)	58.1 (9%)
+ JIT compilation	108.9 (30%)	55.8 (81%)	52.1 (8%)
NumPy forward pass (replaces Torch)	34.3 (12%)	15.1 (53%)	10.1 (2%)
+ normalization folding + numpy input	20.0 (7%)	6.8 (34%)	6.4 (1%)
Environment step	257.5	13.2	567.8
Policy speedup	23×	43×	48×

Table 2: Mean per-step policy latency (microseconds, batch size one) as each optimization is applied in sequence; each row is the fastest pipeline so far, with the policy's share of total step time in parentheses. Policy latencies are means over 30 repetitions of 256 cached observations; the environment step is the mean over 2000 steps of the public NS-Gym example environment (Section 4) under no notification, whose scheduler changes the dynamics only occasionally, unlike the continuous scheduler of Table 1. FrozenLake uses no observation normalization, so its last row reflects only the NumPy input path, and its environment mean is inflated by the rare steps on which the dynamics change and the stock NS-Gym environment regenerates its entire transition table; most steps are far cheaper (95th percentile 233 μs).

4 Results

We evaluate each trained policy on the public NS-Gym example environments, which fix one representative non-stationarity per task. In CartPole the pole mass increases at every step while gravity follows a random walk, and in FrozenLake the surface starts deterministic and grows steadily more slippery as probability mass shifts away from the intended direction. The public Ant environment is configured to decay the torso mass, but MuJoCo rejects every update because the decay would drive the mass non-positive, so this environment is in practice stationary. These public environments are not the full competition benchmark: the final ranking was decided by the organizers on a held-out set of non-stationary configurations that participants never observe (Section 4.1). We report the public environments here as the only externally reproducible measure of agent quality.

For each task we run 100 episodes under each of the three notification levels. Because the submitted policies are notification-agnostic (Section 3.1), their return is invariant to the notification level and differs due to evaluation noise: across the three levels the IQM spans only 6713 to 6721 on Ant, 491 to 493 on CartPole, and is 1.00 throughout on FrozenLake (full per-level breakdown in Appendix B). We therefore pool the 300 episodes per task into the single return reported in Table 3, with a uniformly random policy as a lower bound.

On CartPole and FrozenLake the trained policies come within evaluation noise of the maximum attainable return (IQMs of 492 and 1.0, against ceilings of 500 and 1.0), saturating the public environments; on Ant, which has no fixed performance ceiling, the agent attains roughly 6700 return. All three sit well above the uniformly random baseline that Table 3 includes as a sanity check.

Figure 2 tracks two returns over training: the held-out evaluation IQM used above, and the PLR curriculum return, the episode return the agent earns on the levels the buffer actually plays. On Ant and CartPole the held-out evaluation return rises above the curriculum return: the held-out set samples the level space uniformly, whereas PLR concentrates training on the highest-regret levels, which keeps the curriculum return lower and more volatile even as the policy steadily improves. FrozenLake inverts this ordering: its held-out success rate saturates near 0.5 and sits below the curriculum return. The root cause is that the task is non-Markovian from the agent's perspective: a single observation reveals the current cell but not the transition probabilities, so the agent cannot tell how slippery the surface is, and configurations with sufficiently adverse dynamics are unsolvable. We hypothesize that the inversion then follows from how the PLR score, a sum of advantage magnitudes over an episode, interacts with the state distribution each level induces. When adverse dynamics confine the agent to a small region of the grid, it revisits the same states repeatedly, the

Task	Random	Ours (mean)	Ours (IQM)	Max return
CartPole	22.2	458.5 ± 66.2	492.0	500
FrozenLake	0.00	0.91 ± 0.28	1.00	1.0
Ant	-68.9	6706.5 ± 200.5	6717.3	–

Table 3: Evaluation return on the public NS-Gym environments, pooled over 300 episodes per task (100 episodes at each of the three notification levels). We report the mean with its standard deviation across episodes and the IQM, which discards the top and bottom quartile of episodes and is more robust to outliers. A uniformly random policy is shown as a lower bound, and the maximum attainable return is listed where one exists. CartPole and FrozenLake are at their ceilings up to evaluation noise.

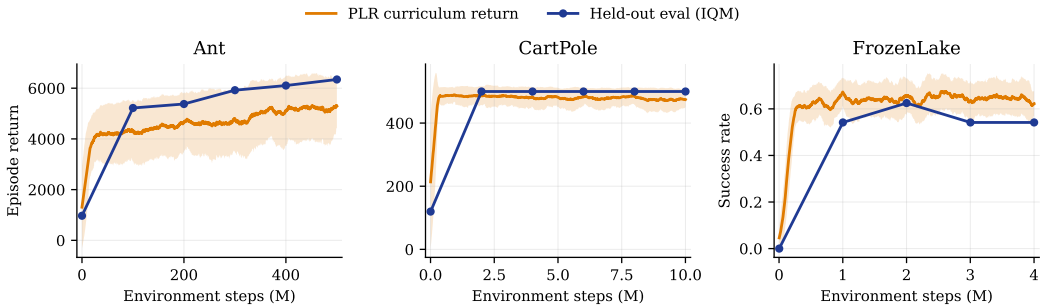


Figure 2: Training curves for the three tasks: the held-out evaluation IQM, measured on configurations sampled from the training level space, and the PLR curriculum return, the episode return on the levels the buffer plays, against environment steps. The curriculum return is shown as a moving average with a shaded band of one rolling standard deviation. These quantities are measured on the training distribution and are not directly comparable to the public-environment returns of Table 3.

critic learns their values accurately, and the per-step advantages there shrink toward zero, so the episode registers little to learn and the level receives a low score. PLR therefore tends to deprioritize these levels, and the ones it does play return more than a uniform held-out sample that includes them.

4.1 Official competition ranking

Our submission dominates the competition leaderboard,[1] ranking first on all four scored axes at once, and is therefore the only non-dominated entry.

The organizers ranked submissions on the hidden benchmark along four axes whose exact scoring functions are not public; we describe each only qualitatively, from the leaderboard documentation, and report the normalized scores as published. Performance is the average undiscounted episodic return under non-stationarity. Adaptability captures how quickly a policy recovers after a change, combining the fraction of runs that cross a recovery threshold, the time taken to converge, and the stability of the recovered return. Resilience captures the immediate return drop at the moment of change and the time to recover from it, rewarding policies that degrade little and recover fast. Efficiency captures the compute cost of adaptation, expressed as a ratio of the extra timesteps and wall-clock time spent. Performance, adaptability, and efficiency are scored without notification, and resilience under full notification.

Table 4 gives the performance ranking, where our agent leads the next submission by 25% in average return. We apply the same model-free recipe, PPO trained with our level curriculum, unchanged across all three tasks, whereas the other leading teams hand-picked a different algorithm for each

[1] Leaderboard results are taken from https://github.com/JayLi1209/ns_gym/blob/feature/tutorial/docs/source/leaderboard.md, which had not yet been merged into the official competition repository at the time of writing.

environment. The margin is comparable for efficiency. The normalized adaptability and resilience components are close across the leading submissions, many clustered near a third (Table 5); our agent still holds the top rank on each of these components, with a decisive margin only on the stability of the recovered return. We are also the only submission whose rank is stable across categories: below us the two runners-up trade places, with pudovikoff leading on performance and adaptability but falling last on resilience, and Kumar20-21 the reverse, rising to second on resilience and efficiency. The other entries trade peak return against robustness and compute cost, while our single recipe leads on all four axes at once.

Rank	Team	FrozenLake	CartPole	Ant	Mean return
1	Ours	PPO	PPO	PPO	2257.2
2	pudovikoff	MLE-replan	DQN	SAC	1800.1
3	Kumar20-21	Dynamic VI	LQR	QR-SAC	1734.3
4	micahr234	MOUSE	MOUSE	–	569.0

Table 4: Performance ranking on the held-out competition benchmark (no-notification mode), as published on the public leaderboard. Mean return is the average undiscounted episodic return under non-stationarity; higher is better. Competitors are identified by their leaderboard handles, and the method used for each task is listed. Our submission deploys the same PPO policy on all three tasks, while the other teams specialized per task; micahr234 submitted no Ant agent (–). These labels are the base learning algorithms as published; the leaderboard does not record what training methodology, such as domain randomization or a curriculum, each team may have used.

Category	Component	Ours	Best competitor
Adaptability	Threshold success (↑)	0.71	0.67
	Convergence time (↓)	0.332	0.333
	Local stability (↓)	6.6×10^{-5}	0.111
Resilience	Immediate drop (↓)	0.333	0.333
	Recovery time (↓)	0.333	0.333
Efficiency	Decision cost (↓)	1.4×10^{-5}	1.9×10^{-5}

Table 5: Component scores behind the adaptability, resilience, and efficiency rankings, as published on the public leaderboard. Scores are organizer-normalized; arrows mark the direction of improvement. The best-competitor column is the strongest value among the other submissions for that component, which may come from different teams. Our submission holds the first-place rank in all three categories. Several components are clustered near one third across the leading submissions, so a few first-place margins are small; the clear separations are in performance (Table 4), efficiency, and the stability of the recovered return. Our near-zero local-stability score reflects that our converged deterministic policies produce near-constant return after a shift, whereas the replanning and value-iteration submissions keep fluctuating as they re-adapt.

5 Conclusion

We posed non-stationary environment configurations as levels and showed that PLR with ACCEL mutations is a simple and effective way to train PPO agents that are robust to unseen non-stationarity. The same model-free recipe, applied unchanged across FrozenLake, CartPole, and Ant, topped every category of the NS-Gym competition and dominated submissions that specialized a separate algorithm or planner for each task. Beyond final return, our optimized inference path kept per-decision compute low on CPU, the cost that the efficiency axis rewards.

6 Limitations

Several limitations qualify these results. Robustness comes entirely from the training-time curriculum: the deployed policy is static and does not adapt online, so configurations whose post-shift

optimum lies far outside the training level distribution remain out of reach. The flip side is that a static policy is cheap to run, which the efficiency axis rewards; coupling the curriculum with lightweight test-time adaptation is a natural next step.

We did not optimize for sample efficiency, since the competition did not score it. The Ant policy, for instance, was trained for 5×10^8 environment steps over roughly a day of wall-clock time on a single desktop CPU; because competitors' training budgets are not published, we cannot say how this compares.

The non-stationarity level space (Appendix C) was designed from intuition about each task's physics, with no principled basis for the particular schedulers, update functions, or argument ranges. A mismatch between this hand-chosen distribution and the competition's hidden non-stationarity would directly hurt transfer.

Finally, we report few ablations. Our aim was to win the competition rather than to isolate each component's contribution, and cost compounds this: the environments run on CPU, where a single Ant training run already takes about a day, making a broad ablation sweep impractical.

ACKNOWLEDGMENTS

This project was carried out as part of the AI Challenges course (WMAI040-10) at the University of Groningen. We thank the organizers of the NS-Gym competition for providing a novel and challenging benchmark and for setting up and running the competition.

REFERENCES

David Abel, André Barreto, Benjamin Van Roy, Doina Precup, Hado van Hasselt, and Satinder Singh. A definition of continual reinforcement learning, 2023. URL https://arxiv.org/abs/2307.11046.

Albert Bou, Matteo Bettini, Sebastian Dittert, Vikash Kumar, Shagun Sodhani, Xiaomeng Yang, Gianni De Fabritiis, and Vincent Moens. TorchRL: A data-driven decision-making library for PyTorch. In *International Conference on Learning Representations*, 2024.

Geonwoo Cho, Jaegyun Im, Jihwan Lee, Hojun Yi, Sejin Kim, and Sundong Kim. TRACED: Transition-aware regret approximation with co-learnability for environment design. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://arxiv.org/abs/2506.19997.

Bruno C. da Silva, Eduardo W. Basso, Ana L. C. Bazzan, and Paulo M. Engel. Dealing with non-stationary environments using context detection. In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pp. 217–224, 2006.

Michael Dennis, Natasha Jaques, Eugene Vinitsky, Alexandre Bayen, Stuart Russell, Andrew Critch, and Sergey Levine. Emergent complexity and zero-shot transfer via unsupervised environment design. In *Advances in Neural Information Processing Systems*, volume 33, pp. 13049–13061, 2020.

Minqi Jiang, Edward Grefenstette, and Tim Rocktäschel. Prioritized level replay. In *International Conference on Machine Learning*, pp. 4940–4950. PMLR, 2021.

Minqi Jiang, Michael Dennis, Jack Parker-Holder, Jakob Foerster, Edward Grefenstette, and Tim Rocktäschel. Replay-guided adversarial environment design, 2022. URL https://arxiv.org/abs/2110.02439.

Nathaniel S. Keplinger, Baiting Luo, Iliyas Bektas, Yunuo Zhang, Kyle Hollins Wray, Aron Laszka, Abhishek Dubey, and Ayan Mukhopadhyay. NS-Gym: Open-source simulation environments and benchmarks for non-stationary markov decision processes. *arXiv preprint arXiv:2501.09646*, 2025. URL https://arxiv.org/abs/2501.09646.

Khimya Khetarpal, Matthew Riemer, Irina Rish, and Doina Precup. Towards continual reinforcement learning: A review and perspectives, 2022. URL https://arxiv.org/abs/2012.13490.

James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13): 3521–3526, March 2017. ISSN 1091-6490. doi: 10.1073/pnas.1611835114. URL https://doi.org/10.1073/pnas.1611835114.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

Arun Mallya and Svetlana Lazebnik. Packnet: Adding multiple tasks to a single network by iterative pruning, 2018. URL https://arxiv.org/abs/1711.05769.

Michael McCloskey and Neal J Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. In *Psychology of learning and motivation*, volume 24, pp. 109–165. Elsevier, 1989.

Arnab Nilim and Laurent El Ghaoui. Robust control of markov decision processes with uncertain transition matrices. *Operations Research*, 53(5):780–798, 2005.

Sindhu Padakandla, Prabuchandran K. J., and Shalabh Bhatnagar. Reinforcement learning algorithm for non-stationary environments. *Applied Intelligence*, 50(11):3590–3606, June 2020. ISSN 1573-7497. doi: 10.1007/s10489-020-01758-5. URL https://doi.org/10.1007/s10489-020-01758-5.

Jack Parker-Holder, Minqi Jiang, Michael Dennis, Mikayel Samvelyan, Jakob Foerster, Edward Grefenstette, and Tim Rocktäschel. Evolving curricula with regret-based environment design. In *International Conference on Machine Learning*, pp. 17473–17498. PMLR, 2022.

Lerrel Pinto, James Davidson, Rahul Sukthankar, and Abhinav Gupta. Robust adversarial reinforcement learning, 2017. URL https://arxiv.org/abs/1703.02702.

Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.

Kate Rakelly, Aurick Zhou, Deirdre Quillen, Chelsea Finn, and Sergey Levine. Efficient off-policy meta-reinforcement learning via probabilistic context variables, 2019. URL https://arxiv.org/abs/1903.08254.

Alexander Rutherford, Michael Beukman, Timon Willi, Bruno Lacerda, Nick Hawes, and Jakob Foerster. No regrets: Investigating and improving regret approximations for curriculum discovery. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL https://arxiv.org/abs/2408.15099.

John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations*, 2016.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. URL https://arxiv.org/abs/1707.06347.

Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world, 2017. URL https://arxiv.org/abs/1703.06907.

Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Arjun KG, Markus Krimmel, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024. URL https://arxiv.org/abs/2407.17032.

Gido M. van de Ven, Nicholas Soures, and Dhireesha Kudithipudi. *Continual learning and catastrophic forgetting*, pp. 153–168. Elsevier, 2025. ISBN 9780443157554. doi: 10.1016/b978-0-443-15754-7.00073-0. URL https://doi.org/10.1016/B978-0-443-15754-7.00073-0.

Tiantian Zhang, Zichuan Lin, Yuxing Wang, Deheng Ye, Qiang Fu, Wei Yang, Xueqian Wang, Bin Liang, Bo Yuan, and Xiu Li. Dynamics-adaptive continual reinforcement learning via progressive contextualization. *IEEE Transactions on Neural Networks and Learning Systems*, 2023. URL https://arxiv.org/abs/2209.00347.

Luisa Zintgraf, Kyriacos Shiarlis, Maximilian Igl, Sebastian Schulze, Yarin Gal, Katja Hofmann, and Shimon Whiteson. Varibad: A very good method for bayes-adaptive deep rl via meta-learning, 2020. URL https://arxiv.org/abs/1910.08348.

A Experimental setup

All experiments, including training, evaluation, and the latency measurements of Sections 3.4 and 3.5, ran on a single desktop machine with an AMD Ryzen 9 5900X processor (12 cores, 24 threads) and 64 GB of DDR4-3200 memory, and no GPU. We ran entirely on CPU by choice rather than requirement, since throughput is bounded by the environment step rather than the small policy network (Section 3.4). The software stack is Ubuntu 22.04 under the Windows Subsystem for Linux (WSL2), Python 3.13, PyTorch 2.9, TorchRL 0.11, NumPy 2.4, Gymnasium 1.2, and NS-Gym 1.0.10.

Table 6 lists the PPO and training-loop settings for each task, and Table 7 the curriculum settings. All agents are optimized with AdamW (Loshchilov & Hutter, 2019) and trained on 24 parallel environments split into two asynchronous collector groups of 12. The curriculum settings were selected by small random searches on Ant and reused for CartPole; FrozenLake was configured separately because its narrower level space favors greedier, less stale replay and no mutation. Across these searches the held-out evaluation return was flat and noisy within the ranges we tried, so rather than read an optimum off the data we picked representative values. Figure 3 shows one such search over the score temperature as an example.

	Ant	CartPole	FrozenLake
Total environment steps	5×10^8	1×10^7	4×10^6
Steps per batch	2560	2560	2560
Mini-batch size	256	256	256
Optimization epochs per batch	5	10	5
Target KL (early stop at $1.5\times$)	0.02	0.02	0.02
Discount γ	0.99	0.99	0.99
GAE λ	0.95	0.95	0.95
Clip range ϵ	0.2	0.2	0.2
Initial learning rate	3×10^{-4}	3×10^{-4}	5×10^{-4}
Final learning rate	3×10^{-5}	3×10^{-5}	1×10^{-5}
Maximum gradient norm	1.0	0.2	0.5
Entropy coefficient	0	0	0.01
Value loss coefficient	0.5	0.5	0.5
Value loss	smooth L1	smooth L1	smooth L1
Hidden layer sizes	256, 256	64, 64	64, 64
Activation	tanh	tanh	tanh
Observation normalization	yes	yes	no
Reward normalization	yes	no	no
Episode horizon	1000	500	200

Table 6: PPO and training-loop hyperparameters per task. The learning rate is annealed linearly from its initial to its final value over the full run.

	Ant	CartPole	FrozenLake
Buffer capacity N	200	100	50
Replay probability p_r	0.3	0.3	0.5
Score temperature β	0.9	0.9	0.1
Staleness coefficient ρ	0.25	0.25	0.1
Mutation probability p_m	0.25	0.25	0.25
Mutation noise scale σ_m	0.2	0.2	0.2
Held-out evaluation levels	16	24	24

Table 7: Curriculum (PLR and ACCEL) hyperparameters per task, with the symbols used in Section 3.3. Replay activates once the buffer holds 10% of its capacity.

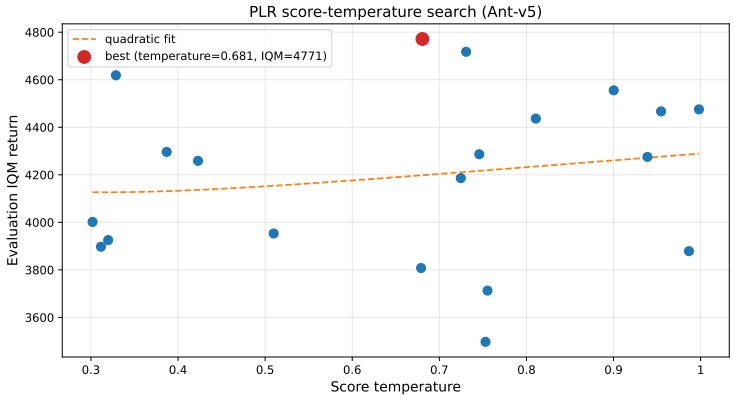


Figure 3: Random search over the PLR score temperature on Ant-v5 (20 trials, 10^7 environment steps each), scored by held-out evaluation IQM return. The quadratic fit is nearly flat, so the search provides little signal and any value in the searched range performs comparably; we use $\beta = 0.9$.

B PER-NOTIFICATION-LEVEL RESULTS

Table 8 breaks the pooled returns of Table 3 down by notification level. Because the submitted policies ignore the notification channel, their return is essentially constant across the three levels; the small differences are evaluation noise over the 100 episodes per cell.

Task	Notification	Mean $\pm$ Std	IQM
CartPole	Full	455.6 ± 69.2	490.7
	Change	459.7 ± 64.3	492.8
	None	460.1 ± 64.8	492.5
FrozenLake	Full	0.90 ± 0.30	1.00
	Change	0.90 ± 0.30	1.00
	None	0.94 ± 0.24	1.00
Ant	Full	6682.2 ± 295.2	6713.2
	Change	6721.8 ± 112.9	6720.7
	None	6715.6 ± 140.9	6716.6

Table 8: Evaluation return on the public NS-Gym environments by notification level (full, change, none), 100 episodes each. The mean is reported with its standard deviation across episodes, alongside the IQM. Returns are stable across notification levels because the submitted policies are notification-agnostic; Table 3 pools these into a single per-task figure.

C LEVEL PARAMETER SPACE

A level assigns to each included tunable parameter a scheduler (Table 9) and an update function (Table 10), each with arguments drawn uniformly from the listed ranges; integer-bounded arguments are sampled as integers. Whenever the scheduler fires at step t, the update function is applied to the parameter p. All six schedulers are eligible for every parameter, while the eligible update functions are restricted per parameter (Table 11). The global argument ranges are calibrated so that the worst-case cumulative change over a 1000-step episode stays bounded even when the scheduler fires every step; for example, the increment step size is capped so that the total drift cannot exceed five units. For several parameters these global ranges are further narrowed where physics demands it, for instance to keep masses positive or gravity negative; the overrides are listed in Table 12. Beyond these constraints, the eligible sets and ranges reflect our intuition about what constitutes plausible non-stationarity in each task. We did not empirically test whether they are optimal, and a mismatch between this training distribution and the non-stationarity used by the competition would directly hurt transfer; we consider this a limitation of our approach.

Qualitatively, the scalar update functions cover four kinds of change. Increment, decrement, deterministic trend, geometric progression, exponential decay, and linear interpolation drift the parameter deterministically: by a fixed step, by a step that grows with time, by a constant factor, by exponential shrinkage, or along a fixed ramp between two values. Random walk and random walk with drift diffuse the parameter stochastically, the latter with a directional bias; the bounded random walk confines this diffusion to an interval, and the Ornstein–Uhlenbeck process pulls the parameter back toward an equilibrium value while it diffuses. The oscillating update perturbs the parameter sinusoidally around its current value. Finally, the three distribution updates act on FrozenLake's transition distribution: the increment and decrement variants move probability mass toward or away from the intended movement direction, making the surface less or more slippery, while uniform drift mixes the distribution toward uniform, the most slippery setting.

All parameters start at the Gymnasium defaults of their task. For FrozenLake the initial transition distribution is itself part of the level: the probability of moving in the intended direction is drawn uniformly from $[0, 1]$ and the remainder is split uniformly at random between the two lateral directions, so training covers near-deterministic through fully slippery dynamics.

Scheduler	Behaviour	Sampled arguments
Continuous	fires every step	–
Periodic	fires every T steps	$T \in [20, 1000]$
Random	fires each step with probability q (frequent)	$q \in [0.01, 0.5]$
Memoryless	same process, geometric gaps, lower rate (rarer)	$q \in [0.001, 0.1]$
Burst	fires for a steps, then is silent for b steps	$a \in [10, 200]$, $b \in [100, 1000]$
Decaying probability	fires with probability $q_0 e^{-ct}$ at step t	$q_0 \in [0.1, 0.9]$, $c \in [10^{-5}, 10^{-3}]$

Table 9: Schedulers and their sampled argument ranges. A scheduler decides at which steps the update function fires.

Update function	Rule	Sampled arguments
Increment	$p \leftarrow p + k$	$k \in [10^{-4}, 5 \times 10^{-3}]$
Decrement	$p \leftarrow p - k$	$k \in [10^{-4}, 5 \times 10^{-3}]$
Exponential decay	$p \leftarrow p\, e^{-ct}$	$c \in [10^{-6}, 10^{-4}]$
Deterministic trend	$p \leftarrow p + ct$	$c \in [10^{-6}, 5 \times 10^{-4}]$
Random walk	$p \leftarrow p + \varepsilon$	$\sigma \in [10^{-3}, 0.05]$
Random walk with drift	$p \leftarrow p + \alpha + \varepsilon$	$\alpha \in [-5, 5] \times 10^{-3}$, $\mu \in [-1, 1] \times 10^{-3}$, $\sigma \in [10^{-3}, 0.05]$
Oscillating	$p \leftarrow p + \delta \sin t$	$\delta \in [0.1, 2.0]$
Geometric progression	$p \leftarrow rp$	$r \in [0.9993, 1.0007]$
Ornstein–Uhlenbeck	$p \leftarrow p + \theta(\mu - p) + \varepsilon$	$\theta \in [10^{-3}, 0.3]$, $\mu \in [-5, 5]$, $\sigma \in [0.01, 0.5]$
Bounded random walk	random walk clipped to $[l, h]$	$\mu \in [-0.1, 0.1]$, $\sigma \in [0.01, 0.5]$, $l \in [-10, 0]$, $h \in [0, 10]$
Linear interpolation	ramp from a to b over T steps	$a, b \in [-10, 10]$, $T \in [50, 1000]$
Distribution increment	$p_1 \leftarrow \min(1, p_1 + k)$, rest split evenly	$k \in [0.005, 0.05]$
Distribution decrement	$p_1 \leftarrow \max(0, p_1 - k)$, rest split evenly	$k \in [0.005, 0.05]$
Uniform drift	$P \leftarrow (1 - \alpha)P + \alpha\, \mathbf{u}$	$\alpha \in [0.001, 0.1]$

Table 10: Update functions with their default (global) argument ranges, applied to parameter p whenever the scheduler fires at step t. Noise is $\varepsilon \sim \mathcal{N}(\mu, \sigma^2)$ with $\mu = 0$ unless listed. The last three rows operate on the FrozenLake transition distribution P; $\mathbf{u}$ denotes the uniform distribution.

Task	Parameter	Inclusion	Eligible update functions
Ant	gravity	1.0	incr., decr., RW, RW drift, trend, osc., OU, BRW, lin. interp.
Ant	torso mass	0.6	incr., decr., exp. decay, geom., RW, OU, BRW, lin. interp.
CartPole	pole mass	1.0	incr., decr., exp. decay, geom., RW, OU, BRW
CartPole	pole length	1.0	incr., decr., RW, OU, BRW
CartPole	cart mass	0.6	incr., decr., RW, OU, BRW
CartPole	gravity	0.6	incr., decr., RW, RW drift, trend, osc., OU, BRW
FrozenLake	transition dist.	1.0	dist. incr., dist. decr., uniform drift

Table 11: Tunable parameters per task, their probability of being included in a sampled level, and their eligible update functions. All six schedulers are eligible for every parameter.

Parameter	Update function	Overridden ranges
Ant gravity	OU	$\theta \in [10^{-3}, 0.1]$, $\mu \in [-15, -3]$
	BRW	$\mu = 0$, $\sigma \in [0.1, 1]$, $l \in [-20, -12]$, $h \in [-5, -1]$
	lin. interp.	$a \in [-12, -8]$, $b \in [-18, -2]$, $T \in [10^4, 5 \times 10^5]$
Ant torso mass	decr.	$k \in [10^{-5}, 10^{-3}]$
	exp. decay	$c \in [10^{-6}, 5 \times 10^{-5}]$
	OU	$\theta \in [10^{-3}, 0.1]$, $\mu \in [0.1, 2]$, $\sigma \in [10^{-3}, 0.1]$
	BRW	$\mu = 0$, $\sigma \in [0.01, 0.1]$, $l \in [0.01, 0.1]$, $h \in [0.5, 5]$
	lin. interp.	$a \in [0.2, 1]$, $b \in [0.05, 3]$, $T \in [100, 1000]$
CartPole pole mass	incr.	$k \in [10^{-3}, 0.1]$
	decr.	$k \in [10^{-5}, 10^{-3}]$
	OU	$\theta \in [10^{-3}, 0.1]$, $\mu \in [0.05, 1]$, $\sigma \in [10^{-3}, 0.05]$
	BRW	$\mu = 0$, $\sigma \in [0.005, 0.05]$, $l \in [0.01, 0.05]$, $h \in [1, 5]$
CartPole pole length	decr.	$k \in [10^{-5}, 10^{-3}]$
	OU	$\theta \in [10^{-3}, 0.1]$, $\mu \in [0.2, 0.8]$, $\sigma \in [10^{-3}, 0.05]$
	BRW	$\mu = 0$, $\sigma \in [0.005, 0.05]$, $l \in [0.1, 0.3]$, $h \in [0.5, 1]$
CartPole cart mass	OU	$\theta \in [10^{-3}, 0.1]$, $\mu \in [0.5, 3]$, $\sigma \in [0.01, 0.1]$
	BRW	$\mu = 0$, $\sigma \in [0.01, 0.1]$, $l \in [0.1, 0.5]$, $h \in [2, 5]$
CartPole gravity	OU	$\theta \in [10^{-3}, 0.1]$, $\mu \in [5, 15]$
	BRW	$\mu = 0$, $\sigma \in [0.05, 0.5]$, $l \in [1, 5]$, $h \in [15, 25]$

Table 12: Parameter-specific argument ranges that override the global defaults of Table 10. Ranges not listed here keep their global values. RW = random walk, BRW = bounded random walk, OU = Ornstein–Uhlenbeck.